

Notebook 1: Transformerの限界とTransolverが必要になる理由

Notebook handout

Source notebook: [Transolver_Learn/japanese_notebooks/01_transformer_bottleneck_ja.ipynb](#)

このPDFはNotebook本文、標準出力、matplotlib図を含む確認用資料です。

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 1

Notebook 1: Transformerの限界とTransolverが必要になる理由

このNotebookでは、Transolverの数式に入る前に、まず「なぜ普通のTransformerをそのままCFD/形状データに使いにくいのか」を確認します。

対象読者は、TransformerやAttentionという言葉聞いたことはあるが、PDE/CFD向けモデルの詳細まではまだ知らない人を想定しています。

このNotebookで学ぶこと

1. Self-Attentionが「点と点の関係」をどう表すか
2. 点数が多い3Dメッシュで、標準Attentionがなぜ重くなるか
3. 物理場では近い点・同じ流れ構造に属する点が重要になりやすいこと
4. TransolverのPhysics-Attentionが、点を直接すべて比較せず、少数の「物理token」を使う発想であること

用語メモ

- Attention: 入力要素同士の「どれをどれくらい参照するか」を重みとして計算する仕組み。
- Self-Attention: 同じ入力の中で、各要素が他の要素を参照するAttention。
- Token: Transformerで処理される単位。自然言語では単語やsubword、CFDではメッシュ点や代表点として考えられる。
- CFD: Computational Fluid Dynamics、数値流体解析。
- PDE: Partial Differential Equation、偏微分方程式。流体や熱、構造などの物理現象を記述する基礎方程式。

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 2

<!-- codex-code-explanation -->

コード解説: 環境準備

このセルでは、数値計算、描画、3Dメッシュ表示に必要なライブラリを読み込みます。PyVista はVTP/ STLのような形状・表面データを扱うためのライブラリです。画面を持たない実行環境でも止まらないように、xvfb の初期化に失敗しても処理を続けます。最後の乱数seed固定は、サンプリング結果や図を毎回ほぼ同じにするためです。

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 1

```
# --- 基本ライブラリを読み込みます。OS操作、数値計算、描画、3Dメッシュ表示を使います。 ---
```

```
import os
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pyvista as pv
```

```
from scipy.spatial.distance import pdist, squareform
```

```
# --- 画面のない環境でも3D描画コードが止まらないようにします。 ---
```

```
# PyVistaをheadless環境で使うための設定。xvfbがない環境では失敗しても続行します。
```

```
try:
```

```
    pv.start_xvfb()
```

```
except Exception as exc:
```

```
    print(f"PyVista xvfb setup skipped: {exc}")
```

```
os.environ.setdefault("XDG_RUNTIME_DIR", "/tmp")
```

```
# --- 乱数を固定し、サンプリングや図が毎回大きく変わらないようにします。 ---
```

```
np.random.seed(42)
```

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 1 output

01_transformer_bottleneck_ja.ipynb:cell3:11: PyVistaDeprecationWarning: This function is deprecated and will be removed

in future version of PyVista. Use vtk with osmesa instead.

PyVista xvfb setup skipped: Please install Xvfb with:

Debian

```
$ sudo apt install libgl1-mesa-glx xvfb
```

CentOS / RHL

```
$ sudo yum install libgl1-mesa-glx xvfb
```

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 4

1. TransformerのSelf-Attentionを物理の例で見る

標準Transformerの中心はSelf-Attentionです。直感的には、各点が「他の点の情報をどれくらい見るべきか」を決めます。

自然言語なら「ある単語が文中の他の単語を参照する」処理です。CFDのsurface meshなら、「ある表面点の圧力や速度を予測するために、別の表面点の情報を参照する」処理に対応します。

ここでは5つの代表点だけを使って、車体まわりの空力相互作用を単純化して見ます。

注釈

- Query: 情報を受け取る側。「この点はどこを見るべきか」と問い合わせる点。
- Key: 参照候補の特徴。「自分はこういう情報を持っている」と示す側。
- Softmax: スコアを合計1の確率のような重みに変換する関数。

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 5

<!-- codex-code-explanation -->

コード解説: Self-Attentionの最小例

ここでは、Self-Attentionを空力的な「どの領域がどの領域を参照するか」という関係として可視化します。physical_scores は手で作った関連度で、softmax によって各行の合計が1になるAttention重みに変換されます。実モデルではこのスコアは学習で決まりますが、ここでは概念理解のために、Bumper、Wakeなどの少数点だけでAttention行列を見えています。

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 2

```
# --- Attentionのスコアを、合計1の重みに変換する関数です。 ---
def softmax(x, axis=-1):
    # 数値的に安定させるため、最大値を引いてからexpを取ります。
    e_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e_x / e_x.sum(axis=axis, keepdims=True)

# --- 少数の代表点だけを使い、Attention行列の意味を見える形にします。 ---
labels = ["Bumper", "Windshield", "Underbody", "Spoiler", "Wake"]
N = len(labels)

# 物理的な結びつきを手で作ったスコア行列。
# 値が大きいほど「その点を強く参照する」と考えます。
# --- 物理的に近い・影響しやすい領域ほど大きいスコアを手で設定します。 ---
physical_scores = np.array([
    [10, 8, 4, 1, 1],
    [8, 10, 5, 2, 2],
    [4, 5, 10, 6, 7],
    [1, 2, 6, 10, 9],
    [8, 7, 6, 9, 10],
], dtype=float)

# --- スコアをsoftmaxし、各QueryがどのKeyを参照するかの重みに変換します。 ---
attention_weights = softmax(physical_scores, axis=-1)

print("Raw score matrix shape:", physical_scores.shape)
print("Attention weight matrix shape:", attention_weights.shape)
print("各行の合計:", attention_weights.sum(axis=1))

for i, label in enumerate(labels):
    print(f"{label:10}: {np.round(attention_weights[i, 3], 3)}")
```

```
# --- Attention重みをヒートマップとして表示します。 ---
plt.figure(figsize=(6, 5))
plt.imshow(attention_weights, cmap="viridis")
plt.title(f"Self-Attention as Aerodynamic Coupling (N={N})")
plt.xticks(range(N), labels, rotation=45)
plt.yticks(range(N), labels)
plt.xlabel("Key: referenced point")
plt.ylabel("Query: target point")
plt.colorbar(label="Attention weight")
plt.tight_layout()
plt.show()
```

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 2 output

Raw score matrix shape: (5, 5)

Attention weight matrix shape: (5, 5)

各行の合計: [1. 1. 1. 1. 1.]

Bumper : [0.879 0.119 0.002 0. 0.]

Windshield: [0.118 0.875 0.006 0. 0.]

Underbody : [0.002 0.006 0.928 0.017 0.046]

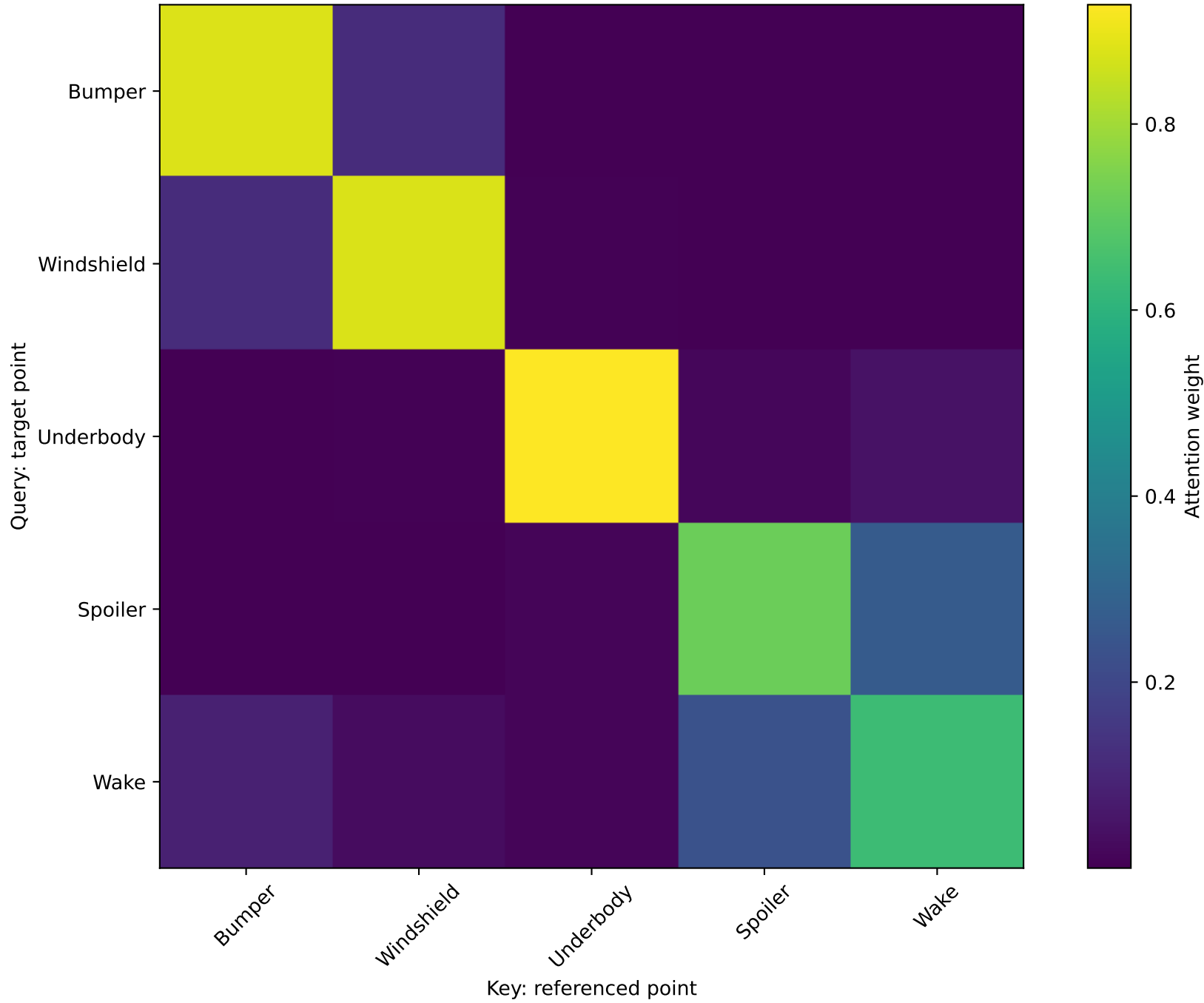
Spoiler : [0. 0. 0.013 0.721 0.265]

Wake : [0.086 0.032 0.012 0.234 0.636]

01_transformer_bottleneck_ja.ipynb:cell6:42: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 2 - Figure 1

Self-Attention as Aerodynamic Coupling (N=5)



Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 7

上の図は、5点だけのAttention行列です。行がQuery、列がKeyです。例えばWakeの行がBumperやSpoilerにも重みを持つなら、後流の状態が前方の形状や後端の流れ制御に影響される、という見方ができます。

重要なのは、標準Self-Attentionでは N個の点すべてについて、N個の点すべてとの関係を計算することです。つまりAttention行列は $N \times N$ になります。

Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 8

2. $O(N^2)$ ボトルネック

標準Attentionの代表的な計算は次の形です。

```
```\text{
scores = Q @ K.T
}```
```

Q と K はどちらも点ごとの特徴から作られる行列です。点数を  $N$  とすると、scores は  $N \times N$  になります。

$O(N^2)$  とは、点数が2倍になると計算量やメモリが約4倍になる、という意味です。これは自然言語の短い文章では許容できても、数十万点の3Dメッシュでは非常に重くなります。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 9

<!-- codex-code-explanation -->

## コード解説: 標準Attentionの計算量

このセルは、点数  $N$  が増えたときに標準TransformerのAttention行列がどれだけ大きくなるかを概算します。Self-Attentionは全点どうしを比較するため、必要な要素数は  $N^2$  です。CFDの表面点や体積格子点は数万から数十万点になりやすいため、この  $O(N^2)$  がTransolverを使う主な動機になります。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 3

```
--- 点数Nごとに、標準Self-Attentionの行列サイズを概算します。 ---
for n in [1_000, 10_000, 100_000, 400_000]:
 # 全点どうしを比較するため、相互作用数はNの2乗になります。
 interactions = n ** 2
 # float32でAttention行列1枚を持つだけの概算メモリ
 memory_gb = interactions * 4 / 1024**3
 print(f"N={n:>7,:}: interactions={interactions/1e9:>8.2f} billion, attention matrix ~= {memory_gb:>8.2f} GB
")
```

## Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 3 output

N= 1,000: interactions= 0.00 billion, attention matrix ~= 0.00 GB

N= 10,000: interactions= 0.10 billion, attention matrix ~= 0.37 GB

N=100,000: interactions= 10.00 billion, attention matrix ~= 37.25 GB

N=400,000: interactions= 160.00 billion, attention matrix ~= 596.05 GB

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 11

400,000点では、Attention行列1枚だけで数百GB規模になります。実際の学習では、head数、batch数、勾配保存、中間activationも必要なので、さらに重くなります。

つまり問題は「Transformerが物理に使えない」 ことではなく、点をそのまますべてtokenとして扱うと、計算のやり方が重すぎる ことです。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 12

## 3. 物理場では全点を同じ密度で見える必要があるのか

流体はNavier-Stokes方程式で記述されるため、厳密には全体が結合しています。ただし、実際の形状表面では次のような偏りがあります。

- 近い点どうしは似た圧力・速度を持ちやすい
- 前縁、後縁、剥離、後流など、物理的に重要な構造がある
- 遠く離れたすべての点を同じ密度で比較するのは非効率な場合が多い

ここではAhmed Bodyデータが手元にあればそれを使い、なければ合成surfaceを作って、距離と圧力類似度の関係を観察します。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 13

<!-- codex-code-explanation -->

## コード解説: 説明用メッシュと物理量の準備

このセルでは、Ahmed Bodyデータがローカルにあればそれを読み込み、なければ説明用の合成surfaceを作ります。ref\_mesh.points が各点の3次元座標、ref\_mesh.point\_data["p"] が圧力に相当する点ごとの値です。後続の解析を軽くするため、全点ではなく最大5000点をランダムに抜き出します。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 4

```
--- Ahmed Bodyがない環境でも説明を進められるよう、合成surfaceを作ります。 ---
```

```
def make_synthetic_surface(n_theta=140, n_z=80):
 """Ahmed Bodyの代わりに使う、説明用の合成surfaceを作ります。"""
 # 円筒状の点群を作るため、角度thetaと高さzの格子を作ります。
 theta = np.linspace(0, 2*np.pi, n_theta, endpoint=False)
 z = np.linspace(-1.0, 1.0, n_z)
 tt, zz = np.meshgrid(theta, z, indexing="ij")
 # 半径にゆらぎを加え、完全な円筒ではない表面形状にします。
 radius = 1.0 + 0.12*np.cos(3*tt) + 0.08*np.sin(2*np.pi*zz)
 x = 2.0*np.cos(tt)
 y = radius*np.sin(tt)
 pts = np.c_[x.ravel(), y.ravel(), zz.ravel()]
```

```
近い場所で滑らかに変わり、後流っぽい領域も持つPressure風の値。
圧力らしい滑らかな分布と、後流らしい低圧領域を人工的に作ります。
```

```
p = (
 1.5*np.cos(tt).ravel()
 + 0.6*np.sin(2*np.pi*zz).ravel()
 - 1.0*np.exp(-((x.ravel() - 2.0)**2 + zz.ravel()**2) / 0.25)
 + 0.08*np.random.randn(pts.shape[0])
)
```

```
cloud = pv.PolyData(pts)
cloud["p"] = p.astype(np.float32)
return cloud
```

```
--- ローカルにAhmed Bodyがあれば読み、なければ上の合成surfaceを使います。 ---
```

```
可能ならAhmed Bodyを使う。なければ合成データへfallback。
```

```
data_dir = "/workspace/transformer/dli/ahmed-body"
```

```
if os.path.isdir(data_dir):
```

```
 vtp_files = sorted(os.path.join(data_dir, f) for f in os.listdir(data_dir) if f.endswith(".vtp"))
```

```
else:
```

```
 vtp_files = []
```

```
if vtp_files:
```

```
 ref_mesh = pv.read(vtp_files[0])
```

```
 data_source = f"Ahmed Body: {vtp_files[0]}"
```

```
else:
```

```
 ref_mesh = make_synthetic_surface()
```

```
 data_source = "Synthetic surface fallback"
```

```
print("Data source:", data_source)
```

```
print("Number of points:", ref_mesh.n_points)
```

```
print("Available point arrays:", list(ref_mesh.point_data.keys()))
```

```
--- 距離行列は重いため、後続解析用に点数を間引きます。 ---
```

```
n_sample_points = min(5000, ref_mesh.n_points)
```

```
ids = np.random.choice(ref_mesh.n_points, n_sample_points, replace=False)
```

```
coords_sample = ref_mesh.points[ids]
```

```
p_sample = np.asarray(ref_mesh.point_data["p"])[ids]
```

```
print("Sampled coordinates shape:", coords_sample.shape)
```

```
print("Sampled pressure shape:", p_sample.shape)
```

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 4 output

Data source: Synthetic surface fallback

Number of points: 11200

Available point arrays: ['p']

Sampled coordinates shape: (5000, 3)

Sampled pressure shape: (5000,)

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 15

## 4. 1例を可視化する

次のセルでは、点群をPressure値で色分けします。Notebook環境によってはPyVistaの3D表示が開かない場合があります。その場合でも、以降の数値解析セルは実行できます。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 16

<!-- codex-code-explanation -->

## コード解説: 圧力場の可視化

このセルは、用意したsurface上の圧力分布を確認するための表示処理です。PyVista で3D表示を試し、実行環境によって難しい場合に備えて matplotlib で2次元投影も描きます。ここではモデル学習はしておらず、物理場データが「点群座標 + 点ごとの物理量」として表現されることを確認します。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 5

```
--- まずPyVistaで3D表示を試します。失敗してもNotebook全体は止めません。 ---
```

```
try:
```

```
 plotter = pv.Plotter()
 plotter.add_mesh(ref_mesh, scalars="p", cmap="coolwarm", show_scalar_bar=True, point_size=4)
 plotter.add_axes()
 plotter.show(title="Surface Pressure Example")
```

```
except Exception as exc:
```

```
 print("PyVista interactive plot skipped:", exc)
```

```
--- 3D表示が使えない環境でも見られるよう、2D投影でも表示します。 ---
```

```
Notebook表示が難しい環境用にmatplotlibでも簡易表示します。
```

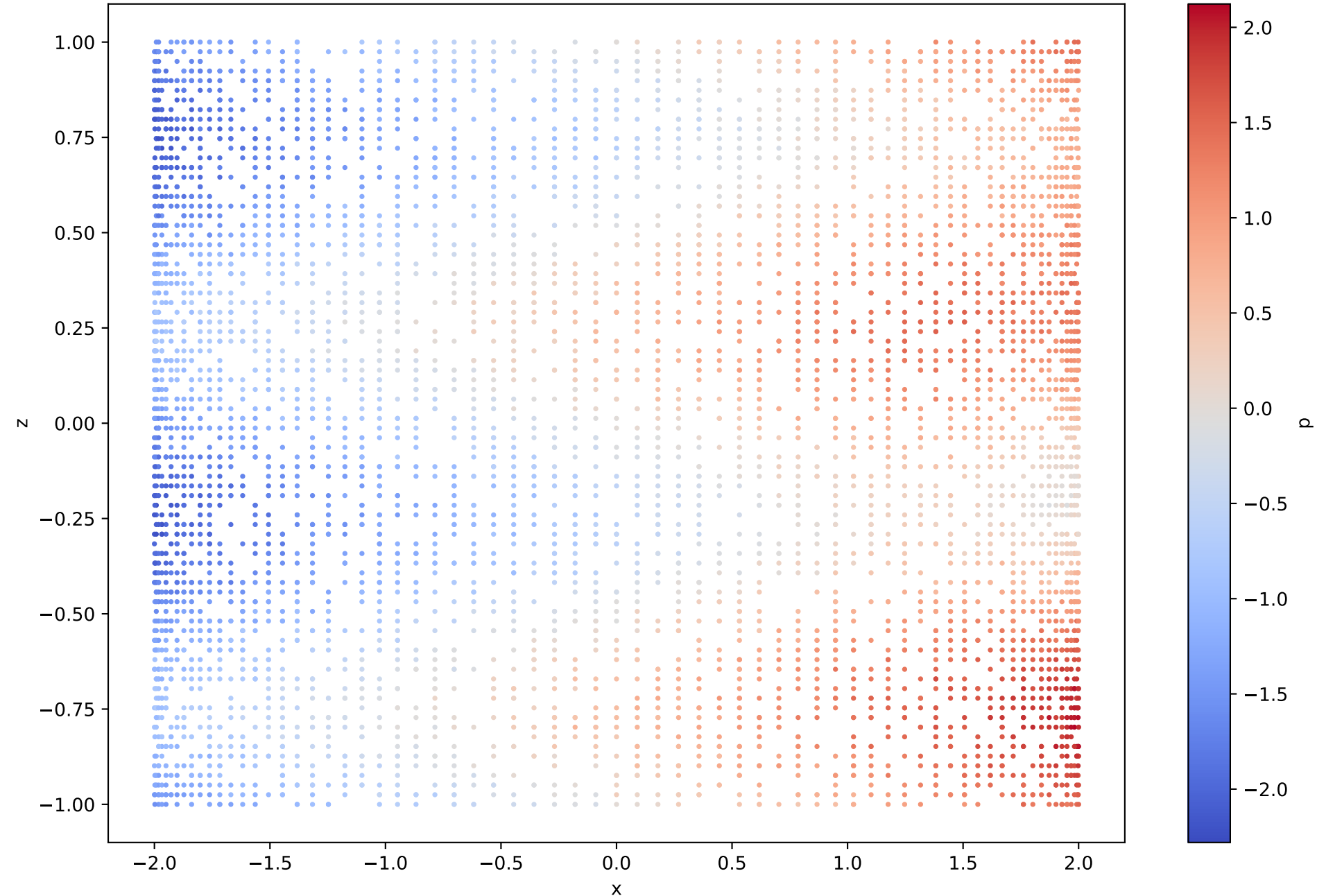
```
plt.figure(figsize=(6, 4))
plt.scatter(coords_sample[:, 0], coords_sample[:, 2], c=p_sample, s=3, cmap="coolwarm")
plt.xlabel("x")
plt.ylabel("z")
plt.title("Pressure sample projection")
plt.colorbar(label="p")
plt.tight_layout()
plt.show()
```

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 5 output

01\_transformer\_bottleneck\_ja.ipynb:cell17:19: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 5 - Figure 1

Pressure sample projection



# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 18

## 5. 距離とPressure類似度を見る

ここでは、サンプル点どうしの距離とPressure類似度を計算します。

注意: これは厳密な物理相関ではなく、1つのsnapshotから作る簡易指標です。ただし、「近い点ほど似た値を持ちやすい」という局所性を直感的に見るには十分です。

用語メモ

- pairwise distance: 点の全組み合わせについて計算した距離。
- similarity: ここでは標準化したPressureどうしの積。似た符号・大きさなら大きくなります。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 19

<!-- codex-code-explanation -->

## コード解説: 距離と物理量の類似性を見る

ここでは、点どうしの距離行列  $D$  と圧力値の類似度  $C_{\text{approx}}$  を作り、距離が近い点ほど圧力も似ているかを調べます。これは厳密なCFD解析ではなく、物理場には局所性や代表構造があることを直感的に見るための処理です。この直感が、全点を直接比較する代わりに代表tokenへ集約するTransolverの発想につながります。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 6

```
--- サンプル点どうしの空間距離を計算します。 ---
print("Calculating pairwise distances...")
D = squareform(pdist(coords_sample))

print("Calculating pressure similarity...")
--- Pressureを標準化し、値の大小ではなく変動パターンを比較しやすくします。 ---
p_norm = (p_sample - np.mean(p_sample)) / (np.std(p_sample) + 1e-12)
p_vec = p_norm.reshape(-1, 1)
C_approx = p_vec @ p_vec.T

triu = np.triu_indices_from(D, k=1)
distances = D[triu]
similarity = C_approx[triu]

--- 距離をbinに分け、距離ごとの平均的なPressure類似度を見ます。 ---
bins = np.linspace(0, np.percentile(distances, 95), 50)
bin_centers = 0.5 * (bins[1:] + bins[:-1])
mean_similarity = []
for i in range(len(bins) - 1):
 mask = (distances >= bins[i]) & (distances < bins[i + 1])
 mean_similarity.append(similarity[mask].mean() if np.any(mask) else np.nan)

mean_similarity = np.asarray(mean_similarity)
mean_similarity_norm = (mean_similarity - np.nanmin(mean_similarity)) / (np.nanmax(mean_similarity) - np.nanmin(mean_similarity) + 1e-12)

--- 距離が近いほど物理量が似る傾向があるかを可視化します。 ---
plt.figure(figsize=(7, 5))
plt.plot(bin_centers, mean_similarity_norm, marker="o", color="tab:blue")
```

```
plt.xlabel("Surface / spatial distance")
plt.ylabel("Mean pressure similarity (normalized)")
plt.title("Locality of physical interactions")
plt.grid(True)
plt.ylim(-0.1, 1.1)
plt.tight_layout()
plt.show()
```

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 6 output

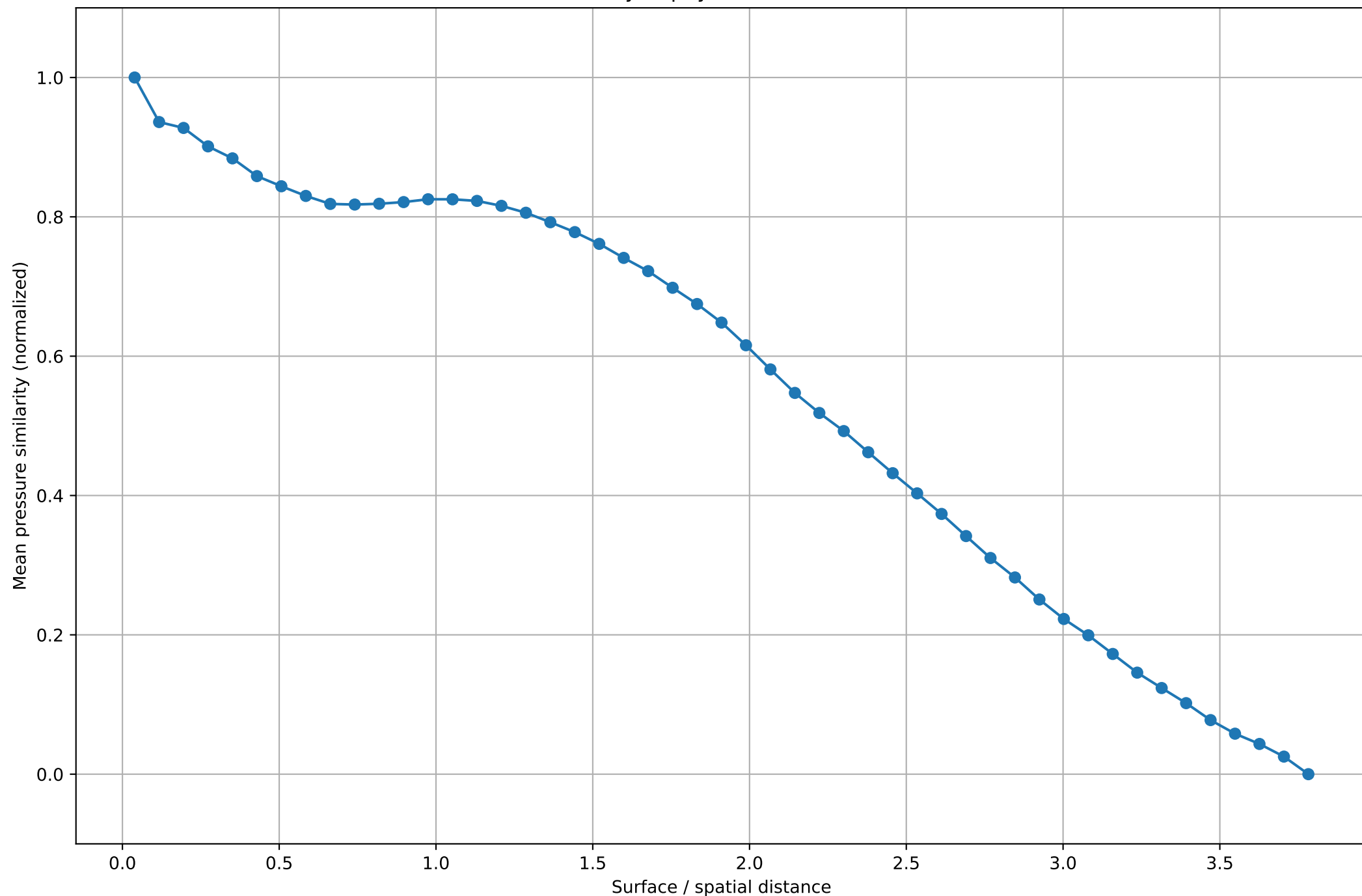
Calculating pairwise distances...

Calculating pressure similarity...

01\_transformer\_bottleneck\_ja.ipynb:cell20:35: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 6 - Figure 1

Locality of physical interactions



# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 21

距離が近いほどPressure類似度が高い傾向が出れば、「すべての点を同じ密度で全比較するより、物理的に意味のある代表状態を使うほうが効率的ではないか」という発想につながります。

これがTransolverの出発点です。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 22

## 6. Transolverの発想: 点から物理tokenへ

Transolverは、すべての点どうしで直接Attentionする代わりに、点群を少数の slice / physics token に集約します。

大まかな流れは次の4段階です。

1. Slice: 各点を少数のsliceへsoftに割り当てる
2. Aggregate: 各sliceに属する点の情報を集約してtokenを作る
3. Attend: 点ではなくtokenどうしでAttentionする
4. Deslice: tokenの更新結果を各点へ戻す

ここではまず、K-means風の単純な空間クラスタリングで「点を少数グループに分ける」イメージだけを見ます。本物のTransolverのsliceは空間クラスタではなく、学習で決まる物理状態へのsoft assignmentです。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 23

<!-- codex-code-explanation -->

## コード解説: 代表sliceの直感を作る

このセルでは、簡易k-meansで点群を  $M=32$  個の領域に分けます。これはTransolver本体のslice学習そのものではありませんが、「多数の点を少数の代表グループにまとめる」という考え方を視覚化するための近似です。実際のTransolverでは、固定クラスタではなく、点の特徴からsoft assignmentとしてslice重みを学習します。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 7

```
--- 説明用に、点群を少数の代表領域へ分ける簡易k-meansを定義します。 ---
```

```
def simple_kmeans(points, n_clusters=16, n_iter=20, seed=42):
 # 初期中心をランダムに選びます。
 rng = np.random.default_rng(seed)
 centers = points[rng.choice(points.shape[0], n_clusters, replace=False)].copy()
 labels = np.zeros(points.shape[0], dtype=np.int64)
 for _ in range(n_iter):
 # 各点から各中心までの距離を計算し、一番近い中心へ割り当てます。
 dist2 = ((points[:, None, :] - centers[None, :, :]) ** 2).sum(axis=-1)
 labels = np.argmin(dist2, axis=1)
 for k in range(n_clusters):
 mask = labels == k
 if np.any(mask):
 centers[k] = points[mask].mean(axis=0)
 return labels
```

```
--- Transolverのsliceに相当する代表グループ数を仮に32にします。 ---
```

```
M = 32
mesh_for_slicing = ref_mesh.copy()
clusters = simple_kmeans(mesh_for_slicing.points, n_clusters=M, n_iter=25)
mesh_for_slicing.point_data["slice_id_proxy"] = clusters
```

```
--- グループ分けされた点群を色で表示します。 ---
```

```
try:
 plotter = pv.Plotter()
 plotter.add_mesh(mesh_for_slicing, scalars="slice_id_proxy", cmap="turbo", show_scalar_bar=False, point_size=4)
 plotter.add_axes()
 plotter.show(title=f"Surface partitioned into M={M} proxy slices")
except Exception as exc:
```

```
print("PyVista plot skipped:", exc)
```

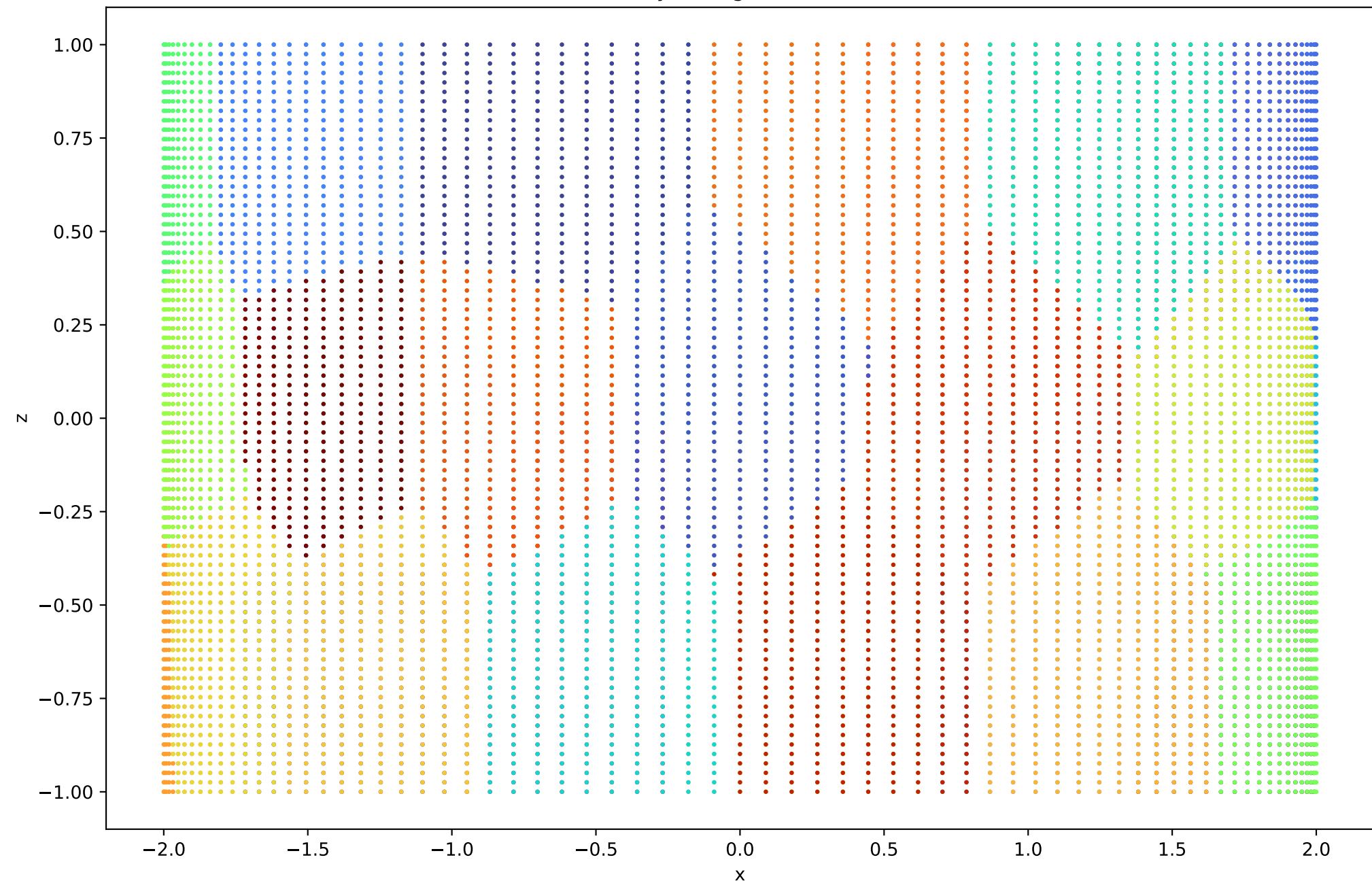
```
plt.figure(figsize=(6, 4))
plt.scatter(mesh_for_slicing.points[:, 0], mesh_for_slicing.points[:, 2], c=clusters, s=2, cmap="turbo")
plt.xlabel("x")
plt.ylabel("z")
plt.title(f"Proxy slicing with $M=\{M\}$ ")
plt.tight_layout()
plt.show()
```

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 7 output

01\_transformer\_bottleneck\_ja.ipynb:cell24:38: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 7 - Figure 1

Proxy slicing with M=32



# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 25

## 7. なぜこれで軽くなるのか

標準Attention:

```
```text
```

点 N 個どうしで比較する $\rightarrow O(N^2)$

```
```
```

TransolverのPhysics-Attention:

```
```text
```

N 点を M 個のslice tokenへ集約する $\rightarrow O(NM)$

M 個のtokenどうしでAttentionする $\rightarrow O(M^2)$

tokenを N 点へ戻す $\rightarrow O(NM)$

```
```
```

通常は  $M \ll N$  です。例えば  $N=400,000$ ,  $M=64$  なら、 $N^2$  と比べてかなり小さくなります。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 26

<!-- codex-code-explanation -->

## コード解説: Transolver的な計算量の見積もり

ここでは、標準Attentionの  $N^2$  と、点数  $N$  をslice数  $M$  に集約して処理するTransolver的な計算量  $2NM + M^2$  を比較します。  $M$  を  $N$  より十分小さく保てると、点数が多いCFDデータでも学習しやすくなります。 重要なのは、計算量を下げながら物理的に必要な相互作用をslice token側に残すことです。

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 8

```
--- 大規模CFD点群を想定し、標準AttentionとTransolver風処理の計算量を比較します。 ---
N = 400_000
for M in [32, 64, 128]:
 # 標準Attentionは全点比較なので N^2 です。
 standard = N^2
 # sliceへ集約して戻す処理を、 $2NM + M^2$ として概算します。
 transolver_like = $2 * N * M + M^2$
 print(f"M={M:>3}: standard N^2 = $\{standard/1e9:>8.1f\}$ B, Transolver-like= $\{transolver_like/1e6:>8.1f\}$ M, ratio=
{standar
d/transolver_like:>8.1f}x")
```

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Code 8 output

M= 32: standard  $N^2$ = 160.0B, Transolver-like= 25.6M, ratio= 6249.8x

M= 64: standard  $N^2$ = 160.0B, Transolver-like= 51.2M, ratio= 3124.8x

M=128: standard  $N^2$ = 160.0B, Transolver-like= 102.4M, ratio= 1562.3x

# Notebook 1: Transformerの限界とTransolverが必要になる理由 / Markdown 28

## まとめ

このNotebookで確認したことは次の通りです。

1. 標準Transformerは、各tokenが全tokenを見るSelf-Attentionを使う。
2. メッシュ点をtokenとして扱くと、Attention行列は  $N \times N$  になり、点数が多いCFDでは重すぎる。
3. 物理場には局所性や代表的な流れ構造があるため、全点を直接比較する以外の表現が考えられる。
4. Transolverは、点を少数のphysics-aware tokenへ集約し、そのtoken上でAttentionを行う。
5. 次のNotebookでは、このSlice/Aggregate/Attend/DesliceをNumPyで実装し、Transolverの中身をより具体的に理解する。